SHIELD-AI

A Formally-Verified Online Learning Controller for Safe Kubernetes Auto-Scaling and Self-Healing

[Author Name]

[Supervisor Name]

M.Tech Thesis Defense · Day 18 · 2026-09-01

The locked thesis sentence

> *"Naive ML-based Kubernetes controllers are unsafe under burst load. SHIELD-AI combines online ML (River) with a formally-verified safety shield (TLA+) to retain ML adaptability while provably satisfying safety invariants that bare controllers violate."*

Source: tasks/THESIS.md:3-7

Why now? The Day-15 motivating failure

	Scaling lag (s)	Error rate (%)	Replicas (start → end)
	15	0.0	2 → 6
	5	0.0	2 → 2
eld)	90	69.2	2 → 2

The ML-only operator was WORSE than HPA under burst load: it emitted a single heal action and never scaled.

Source: docs/thesis/02_introduction.md:30-37; docs/thesis/07_results.md

Two defects caused the Day-15 failure

- Ordering bug: decision engine checked heal before scale. Under burst load, anomaly detector flagged high p95 as anomalous, engine emitted heal, scale was never called.
- No online learning: _run_online never called .learn_one(). The Day-7 offline model was frozen for 8 days, never adapting to live traffic.
- Combined effect: ML-only controller stuck at 2 replicas with 69% error rate while HPA scaled correctly.

Source: src/decision/decision_engine.py:50-95 (P1 fix)

SHIELD-AI architecture

- Prometheus (10 s) → Kafka (k8s-metrics) → Faust 30-s windows
- → Kafka (k8s-features) → Decision Engine (River HTR + HalfSpaceTrees)
- → Safety Shield (TLA+ verified, 6 invariants) → Kafka (k8s-decisions)
- → Operator (Kafka consumer + K8s actuator, re-validates)
- → workload-v2 Deployment (scale or heal)

Source: docs/thesis/05_proposed_system.md:9-51; docs/paper/main.tex §V

Why River Hoeffding Adaptive Tree Regressor?

- Online learning on a stream: one-pass, O(memory), no GPU, no minibatch
- Concept drift: tree adapts split criteria online; a frozen neural net does not
- Explainability: leave-one-out perturbation gives defensible answer to "why did the controller scale here?"
- Rejected alternatives:
- - scikit-learn Random Forest (offline; would need full retrain on drift)
- - PyTorch neural net (GPU unavailable on single-node kind cluster)
- - RL (PPO on replica count; out of M.Tech scope, needs simulation env)

Source: docs/thesis/05_proposed_system.md:115-130

Decision rule — load-first ordering (P1 fix)

```
if predicted_replicas != current_replicas:
action = 'scale' # load is dominant
elif anomaly_score > 2 * threshold:
action = 'heal'
else:
action = 'noop'

# after every decision:
engine.learn(features, current_replicas) # online learn loop
```

Source: src/decision/decision_engine.py:50-95

Safety Shield — the trust boundary

Invariant	What it enforces
SafetyMinReplicas	replicas ≥ 1
SafetyMaxReplicas	replicas ≤ 10
SafetyScalingStep	|delta| ≤ 2 per decision
SafetyHealNoScale	heal preserves replicas
SafetyBoundedRate	cooldown elapsed or no action pending
LivenessEventuallyScaleUp	sustained demand → scale eventually

Source: specs/SafetyShield.tla:230-290; src/safety/safety_shield.py

ML+Shield composition — the central paper claim

"The closed-loop system is safe iff the shield is safe, regardless of ML oracle behavior."

- ML oracle modeled as thin non-deterministic abstraction: any (action, target) in 0..ML_OUTPUT_RANGE, including out-of-bounds and over-large-step.
- Two parallel paths in one module: SHIELD (production) and ML_Only (the bug).
- TLC verifies all six shield invariants hold on every reachable state of the joint spec AND the ML_Only path CAN violate MlSafetyMaxReplicas (3-step counterexample).
- Result: shield is NECESSARY and SUFFICIENT for safety.

Source: specs/ML_Composition.tla; TLC ~273K states, ~4 min

Shield in action — offline replay (225 decisions)

Outcome	Count	%
Allowed unchanged	178	79.1%
Clamped (|delta|>2)	47	20.9%
Rejected	0	0.0%
Total	225	100%

- All 47 clamps are 5-10 → 4 via max_scale_step=2.
- Shield never had to reject: ML already conservative (predicts 7 not 10).
- But clamping is the safety guarantee — no over-large step reaches K8s.

Source: scripts/replay_shield.py; data/features_v2.csv

FIRM-style threshold baseline (P2)

	AI predicted	FIRM predicted	Actual target
	7 × 84, 1 × 1	10 × 84, 2 × 1	10 × 84, 2 × 1
	7 × 84, 1 × 1	10 × 84, 2 × 1	10 × 84, 2 × 1
	5 × 54, 1 × 1	5 × 49, 4 × 5, 2 × 1	5 × 49, 4 × 5, 2 × 1

- FIRM hits target exactly: hand-tuned thresholds align with the heuristic.
- AI predicts 7 not 10: under-predicts by 3, but in the right direction.
- Trade-off: AI has room to improve on accuracy; FIRM is fixed.
- The Safety Shield closes the safety gap regardless.

Source: src/baselines/firm_controller.py; data/features_v2.csv

Why TLA+, not just unit tests?

- Unit tests: verify that the code passes test cases.
- TLA+: verify that the code is correct for every possible input (within bounds).
- Difference: 'tested' vs 'verified' — the shield's invariants hold on every reachable state, not just on the sample the tests happen to cover.
- Composition spec extends this: shield's safety holds for every possible ML oracle output, not just the ones in our test set.
- Industrial precedent: Amazon (DynamoDB, S3), Microsoft (Cosmos DB).

Source: docs/thesis/03_literature_survey.md:30-50

Why Kafka, not in-memory bus?

- Durability: if decision engine crashes, no metrics are lost.
- Audit trail: every decision is on a Kafka topic (logs/decisions.log).
- Replay: offline replay (scripts/replay_shield.py) reads from the same topic.
- Cost: ~10 ms per hop latency. Trade-off accepted for the safety claim.
- Alternative rejected: in-memory queue — no audit, no replay, no recovery.

Source: docs/thesis/08_discussion.md:8-15

Why Kafka actuator, not Kopf CRD handler?

- Simpler test surface: one process to test, not two (Kopf + CRD lifecycle).
- No requeue logic, no watcher, no CRD reconciliation.
- Decouples the operator from the cluster's CRD registry.
- Easier to formally model: operator's input is a Kafka topic.
- Decision documented in AMENDMENTS 2026-08-23.

Source: src/kopf_operator/actuator.py; tasks/AMENDMENTS.md

Reproducibility — the single-command demo

```
# On a fresh VM:
git clone git@github.com-personal:sudo-Harshk/k8-auto-scaling-self-healing.git
cd k8-auto-scaling-self-healing
make build-image && make load-image
make deploy-kafka deploy-prometheus deploy-workload
make pipeline-up
make load-baseline && make load-burst && make load-rampdown
make stats
make tla # ~30 sec
make tla-composition # ~4 min
```

Source: Makefile; docs/GOLDEN_RUN.md

Tests & statistical rigor

- 53 unit tests pass on every commit (under 1 second).
- 17 anti-drift tests on the Safety Shield — each intentionally violates one invariant and verifies the class catches it.
- N=3 head-to-head evaluation (Day 14/15) plus per-scenario FIRM comparison.
- N≥10 harness ready (scripts/eval/run_N10.sh + stats_report.py).
- Effect sizes with Cohen's d and 95% CI on every comparison metric.

Source: tests/; scripts/eval/

Threats to validity

- Single-node kind cluster (no multi-node scheduling realism).
- Single workload class (workload-v2, DB-backed Flask + SQLite).
- 60-second cooldown limits scaling agility.
- Detection rate 55-65% on organic baseline (anomaly detector tuned for fault-injection patterns).
- N≥10 not yet executed; current eval is N=3 + per-scenario FIRM.
- Single Kafka as point of failure.

Source: docs/thesis/07_results.md (Threats); docs/paper/main.tex §IX

Where we sit vs HPA, KEDA, FIRM

	KEDA	FIRM	SHIE
	■	■	■
	■	■	■
	■	■	■ 6 ir
	■	■	■ Riv
	■	■	■

Source: docs/thesis/03_literature_survey.md:55-60

Three contributions (recap)

- Hybrid ML + formal safety controller — K8s operator whose action space is the intersection of ML-driven decisions and a TLA+-verified invariant set.
- Empirically-validated failure mode of pure ML controllers — Day-15 N=3 evidence showing AI without the shield gets stuck at 2 replicas with 69% error (the motivating failure).
- Reproducible artifact — 53 unit tests, containerized make demo, full TLA+ TLC trace, FIRM-style ML baseline.

Source: tasks/THESIS.md:9-13

Future work & questions

- Production deployment: multi-node cluster, multi-tenant policies, HA Kafka.
- Concept drift monitoring: ADWIN on residual error of the replica predictor.
- Tighter TLA+: probabilistic safety bounds, multi-tenant fairness, leader-election fault tolerance.
- Beyond K8s: GPU allocation, edge workload placement, network QoS — same architecture (Kafka + Faust + River + TLA+ shield).
-
- Questions?

Source: docs/thesis/09_conclusion.md:51-65